

TEAM 5 | YCC

안전하고 편리하게
기업을 위한 인사 관리 시스템

HR BANK

목차

기업을 위한 인사관리 시스템
HR BANK

01 개요

프로젝트 개요

02 기획

주제 선정 이유
해결방법
프로젝트 목표
SWOT 분석
기술 스택 및 개발 환경
개발 일정

03 요구사항 및 UI

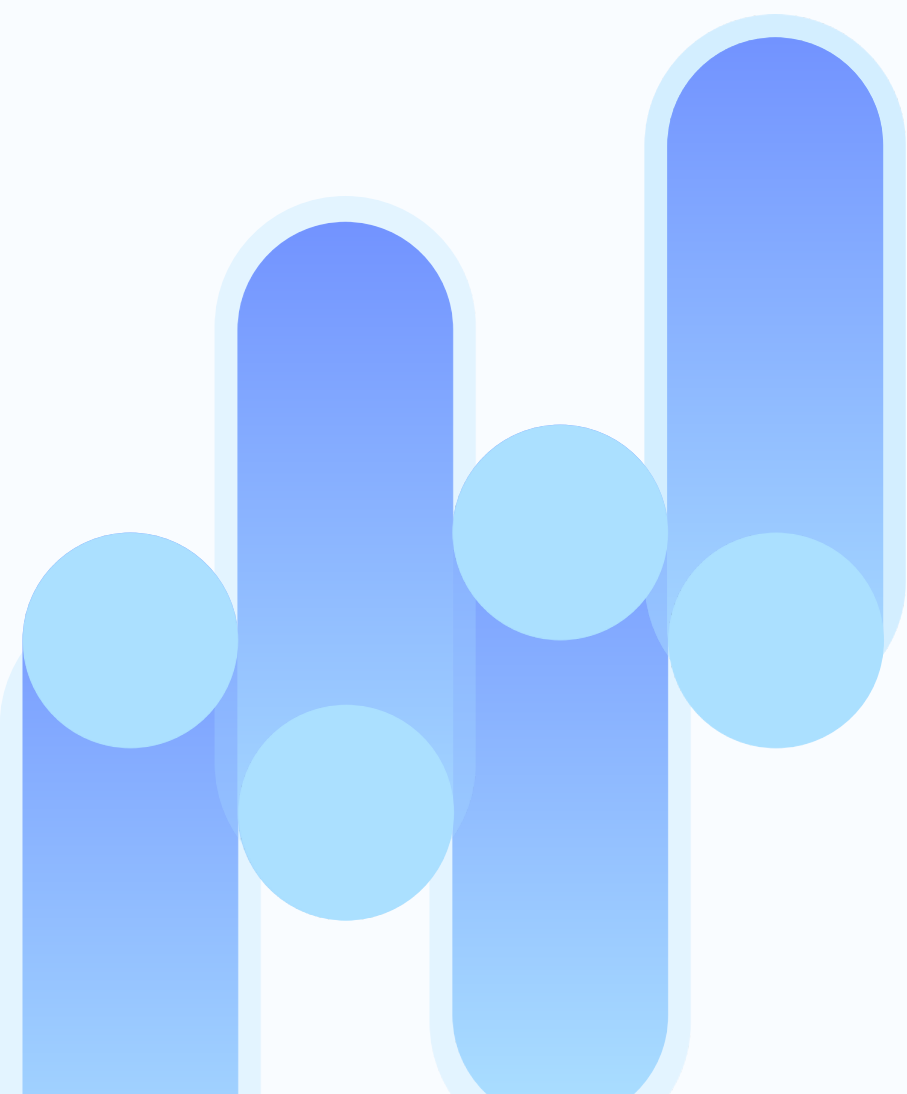
기능리스트
업무 분담
협업 방식
UI 흐름도

04 설계

ERD
시스템 아키텍처

05 구현

구현 페이지
트러블 슈팅



HR BANK

HR Bank는 배치 기술을 활용해 데이터를 자동으로 관리하는 개방형 HR 플랫폼입니다. 직원 정보와 인사 데이터를 안전하게 저장하고 대량의 데이터를 주기적으로 백업하여 효율적이고 안정적인 인사 관리 환경을 제공합니다.

Period

2026. 05. 14. - 2026. 05. 26.

Members

장현우 오소현 석지에 송유정 김수아

Function

부서 관리

부서 등록·수정·삭제·목록 조회

직원 정보 관리

직원 등록
정보 수정·삭제·목록 및 상세 조회

파일 관리

파일 저장·다운로드

직원 정보 수정 이력 관리

이력 등록·목록 조회
이력 상세 변경 내용 조회

데이터 백업 관리

대시보드

Tool, Framework & Language

Framework
Spring Boot



Database
PostgreSQL



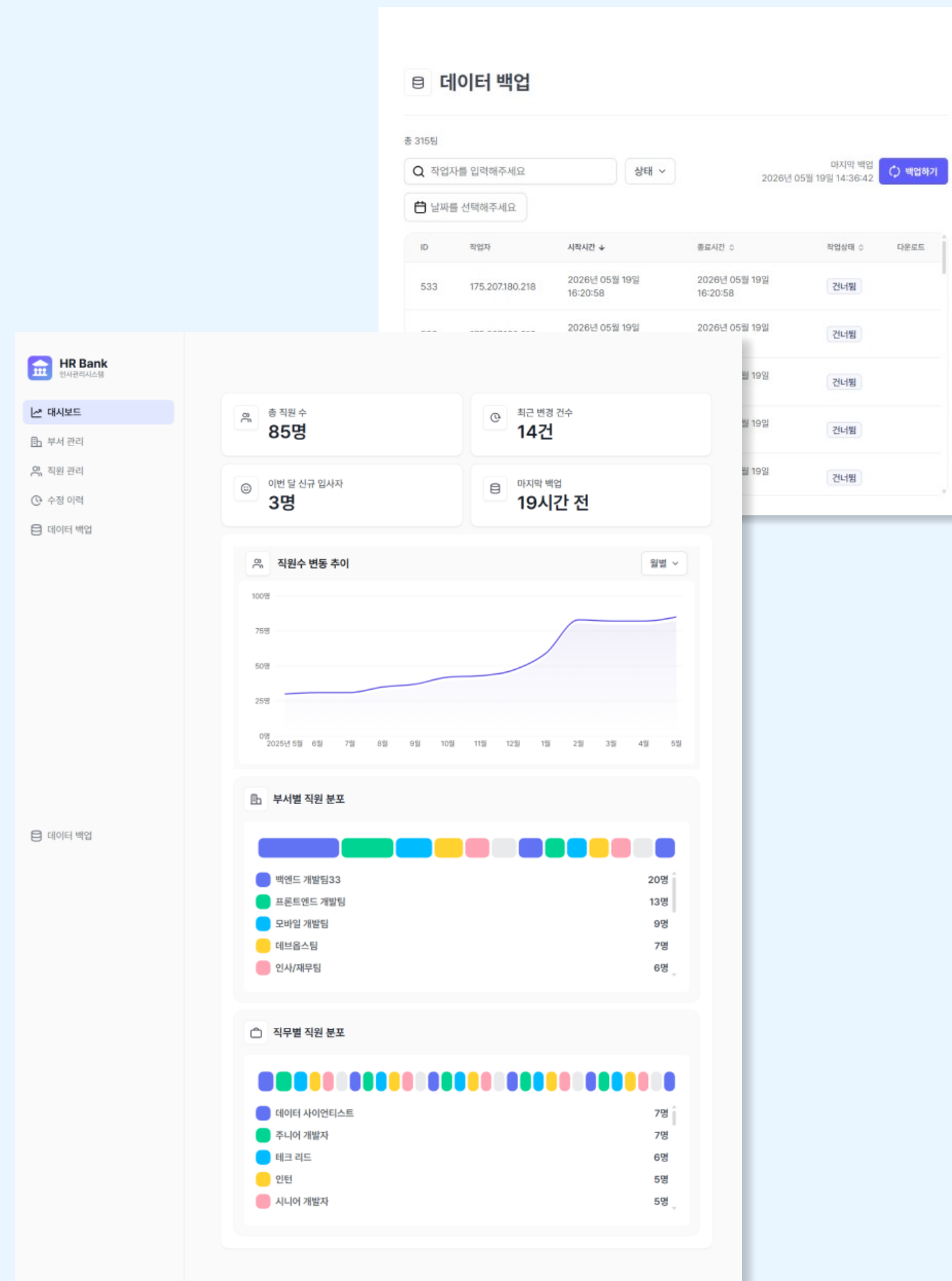
Spring Backend
JPA,
QueryDSL
Spring Scheduler
MapStruct



Documentation
springdoc-openapi (Swagger)



배포 및 협업
Railway.io / Git & GitHub



주제 선정 이유

기존 인사 시스템은 대용량 데이터 처리 시 메모리 부족(OOM) 장애가 빈번하고, 데이터 변경 이력이 누락되어 신뢰성이 낮다는 문제점이 있습니다. 따라서 데이터의 안정적인 운영과 투명한 추적 관리가 필수적인 인사 데이터 관리의 본질을 해결하고자 **HR BANK 프로젝트**를 선정하게 되었습니다.



실시간 대용량 처리로 인한 시스템 부하



데이터 변경 이력 관리의 부재



대규모 데이터 적재 시 서버 자원 고갈 (OOM 발생)



HR BANK

해결방법

AS - IS

대량의 인사 데이터 변경 시 실시간 처리에 따른
서버 부하 및 시스템 응답 속도 저하



데이터 변경 이력 미기록으로 인한
데이터 정제 오류 추적의 어려움



대용량 데이터 일괄 적재로 인한 서버 자원 고갈 및
OOM(메모리 부족) 발생



TO - BE

Batch 처리를 통한 부하 분산,
Audit 기록을 통한 정합성 확보 및
메모리 최적화로 시스템 안정성 달성

Audit 체계 및 Batch형 아키텍처 구축을 통한
데이터 이력 추적 체계 확립

데이터 분할 처리 구현을 통한
메모리 부하 해소 및 안정적인 운영 환경 구축

프로젝트 목표



대규모 데이터 처리의 안정성 확보

실시간 트래픽을 피한 주기적 스케줄링을 통해
시스템 부하를 분산하고
서비스 가용성을 극대화합니다.



데이터 무결성 및 추적성 강화

데이터 정제 및 수정 시 발생할 수 있는 오류를 방지하기
위해 이력을 관리하고, 유실 시 **즉각적인 추적이 가능한**
신뢰 구조를 확보합니다.



서버 자원 사용의 최적화 (OOM 방지)

제한된 서버 자원 안에서 전체 데이터를 한 번에 메모리에
적재할 때 발생하는 **OOM(Out Of Memory)**
문제를 근본적으로 차단합니다.

HR BANK

SWOT 분석

Strengths 강점

- 데이터 무결성 및 대용량 데이터 처리를 고려한 PostgreSQL 기반 실무 환경 반영
- Chunk 처리로 OOM 방지
- Audit 및 백업 기능으로 데이터 안정성 확보

Weaknesses 약점

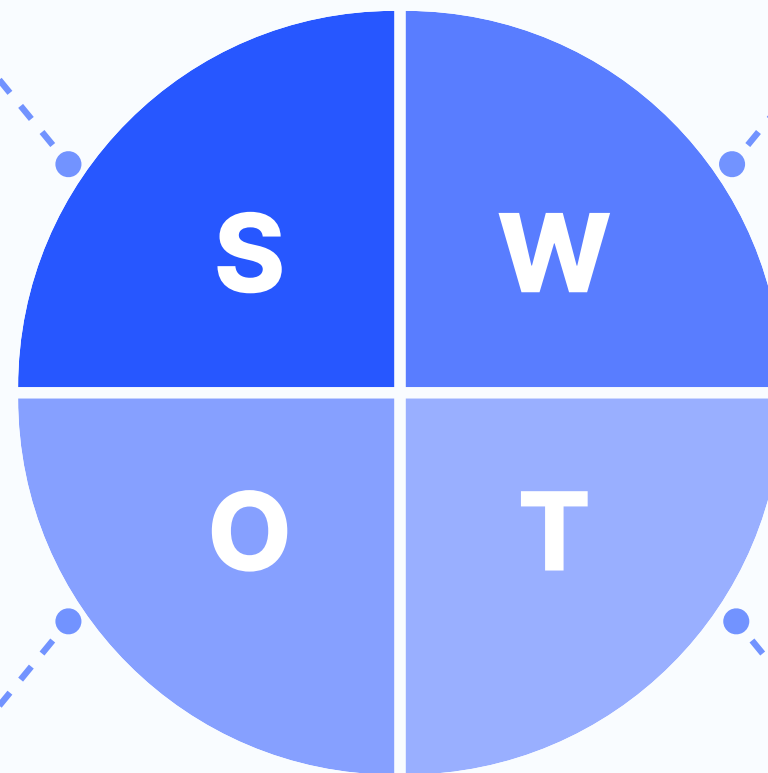
- 실시간 반영보다 주기적 처리에 의존
- 배치 주기 증가 시 자원 사용량 증가

Opportunities 기회

- 대용량 데이터 및 신규 기능 확장 가능
- 실무형 Batch 아키텍처 경험 확보

Threats 위협

- 데이터 증가에 따른 성능 저하 가능성
- 배치 실패 및 스케줄러 장애 시 운영 리스크 존재



기술 스택 및 개발 환경

Framework



Spring Boot



Spring JPA



QueryDSL



MapStruct

DB

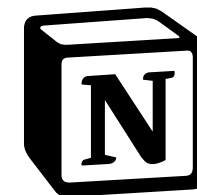


PostgreSQL

문서화



Swagger



Notion

배포 및 협업



Railway



Git



GitHub

개발 일정

Period

2026. 05. 14. - 2026. 05. 26. (총 13일)



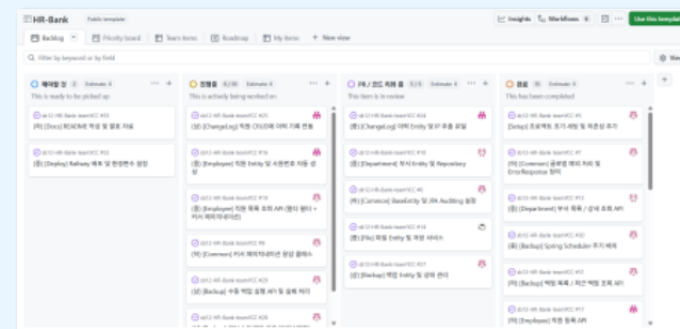
Notion

회의록 및
프로젝트 문서화



Dicord

이슈 공유 및
검토를 위한 실시간
소통, 회의 진행



Git

일정관리 및 업무 분담,
진행과정 공유



기능 리스트

인사 관리 시스템

직원 관리

직원 등록 · 조회 · 수정 · 삭제

사번 자동 부여

이메일 중복 방지

퇴사 시 수정 처리

프로필 이미지 삭제 연동

부서 관리

부서 등록 · 조회 · 수정 · 삭제

부서명 중복 방지

빈 부서 삭제 제한

파일관리

파일 다운로드

메타 DB 저장

파일 로컬 저장

수정 이력 관리

수정 이력 조회

IP 자동 추출

수정값 전/후 저장

목록 조회 시 상세 내용 미포함

데이터 백업 관리

데이터 백업 생성 · 조회

실패 시 로그 전환

1시간 주기 스케줄링

OOM 방지 CSV 파일 저장

기능 리스트

프로젝트의 전체 기능을 체계적으로 정리하고 문서화하여 개발 방향성을 명확히 설정했습니다.

API 명세서

<

API 명세서

RESTful API 설계

- 엔드포인트 정의
- 요청/응답 구조
- 에러 정의

기능 명세서

ID	기능 분류	Aa 기능	설명	입력 정보	우선순위	상태	비고
EMP-1	직원 관리	직원 목록 조회	직원 목록을 조회합니다.	이름 또는 이메일(nameOrEmail), 사번(employeeNumber), 부서이름(departmentName), 직함(position), 입사일(hireDateFrom), 퇴사일(hireDateTo), 재직 상태(status), 이전 페이지 마지막 요소 ID(idAfter), 커서(cursor), 페이지 크기(size), 정렬 필드(sortField), 정렬 방향(sortDirection)	높음	● 진행 중	이름-이메일-부서-직함-사원번호는 부분 일치 조건입니다. (EMP-14) 입사일은 범위 조건입니다. (EMP-14) 상태는 완전 일치 조건입니다. (EMP-14) 조회 조건이 여러 개인 경우 모든 조건을 만족한 결과로 조회합니다. (EMP-14) 정렬은 이름-입사일-사원번호 중 하나만 선택할 수 있습니다. (EMP-15) 페이지네이션은 이전 페이지 마지막 요소 ID와 커서를 활용합니다. (EMP-15)
EMP-2	직원 관리	직원 등록	이름, 이메일, 부서, 직함, 입사일, 프로필 이미지를 통해 새로운 직원을 등록합니다	직원 데이터(JSON), 프로필 이미지(binary)	높음	● 진행 완료	이메일은 중복 불가합니다. (EMP-9) 사원 번호는 자동으로 부여됩니다. ((EMP-10) 상태는 자동으로 재직중 으로 초기화됩니다. ((EMP-11) 프로필 이미지는 선택적으로 등록할 수 있습니다. (EMP-12) 프로필 이미지는 파일 관리 요구사항에 따라 등록합니다. (EMP-12)
EMP-3	직원 관리	직원 상세 조회	직원 상세 정보를 조회합니다.	직원 ID (path)	높음	● 진행 중	단일 직원의 모든 정보를 반환합니다.
EMP-4	직원 관리	직원 삭제	직원을 삭제합니다.	직원 ID (path)	높음	● 진행 중	삭제 시 프로필 이미지도 함께 삭제됩니다. (EMP-12) 퇴사 상태는 삭제가 아닌 수정으로 처리합니다. (EMP-13)
EMP-5	직원 관리	직원 수정	사원 번호를 제외한 모든 속성을 수정할 수 있습니다.	직원 ID (path), 직원 데이터(JSON), 프로필 이미지(binary)	높음	● 진행 중	이메일은 중복 불가합니다. (EMP-9) 퇴사 처리 시 삭제가 아닌 상태 변경으로 처리합니다. (EMP-13)
EMP-6	직원 관리	직원 수 추이 조회	지정된 기간 및 시간 단위로 그룹화된 직원 수 추이를 조회합니다.	시작 일시(from), 종료 일시(to), 시간 단위(unit)	중간	● 진행 중	파라미터를 제공하지 않으면 최근 12개월 데이터를 월 단위로 반환합니다.
EMP-7	직원 관리	직원 분포 조회	지정된 기준으로 그룹화된 직원 분포를 조회합니다.	그룹화 기준(groupBy), 직원 상태(status)	중간	● 진행 예정	부서별 또는 직무별 직원 분포를 조회합니다.
EMP-8	직원 관리	직원 수 조회	지정된 조건에 맞는 직원 수를 조회합니다.	직원 상태(status), 입사일(fromDate), 퇴사일(toDate)	중간	● 진행 예정	상태 필터링 및 입사일 기간 필터링 할 수 있습니다. (EMP-15)
EMP-9	직원 관리	이메일 중복 검증	직원 등록 및 수정 시 이메일 중복 여부를 검증합니다.	(내부 로직) 이메일	중간	● 진행 중	이메일은 다른 직원과 중복될 수 없습니다.

기능 명세서

- 부서 관리, 직원 정보 관리
- 파일 관리 기능
- 이력 관리 및 백업 기능
- 대시보드 구성

업무 분담

5팀 YCC

Codeit Sprint Team Members



장현우 팀장

프로젝트 기술 리드
공통 인프라·데이터 백업·기능 통합



김수아

포트폴리오, 문서화
스케줄링, 배포



송유정

파일 업로드·다운로드
파일 메타데이터 저장·조회

HR Bank 프로젝트 팀원들을 소개합니다

우리는 Young Creator Crew 입니다.



석지예

기능 명세서 및 API 명세서
부서 기능



오소현

ERD 작성 및 DB 모델링
직원·직원 수정 이력

협업 방식

GitHub Strategy

개인별 dev 브랜치에서 기능 개발
작업 완료 후 Pull Request 생성
코드 리뷰 후 Dev 브랜치로 Merge
충돌 발생 시 담당자 간 조율 후 해결

Code Review & Convention

PR 기반 코드 리뷰
Layered Architecture 기준 패키지 분리
DTO / Entity / Mapper 역할 분리
ErrorCode 기반 예외 처리
테스트 및 빌드 결과 확인 후 Merge

Issue Tracking

GitHub Project로 작업 이슈 관리
데일리 스크럼 - 오전, 오후 진행

Collaboration Tools

05.18 3일차 회의록

- HR Bank 프로젝트**
날짜 2025. 5. 18 (월)
팀장 장현우
참여자 장현우 · 오소현 · 석지예
- 개발 분담 방식**
- 레이어 기반 분담 방식. → controller, service, entity 등
 - 기능 기반 분담 방식. → 부서, 백업, 직원 등
 - 이슈 기반 분담 방식. → 깃허브 프로젝트 참조.
 - 혼합형 (Entity, Dto) → 공통 구조 설계 + 레이어 기반 방식

Notion

회의록, API 문서, 요구사항 정리

Discord

실시간 소통, 작업 이슈 공유

이슈 트래킹 및 진행 상태 관리

GitHub Project

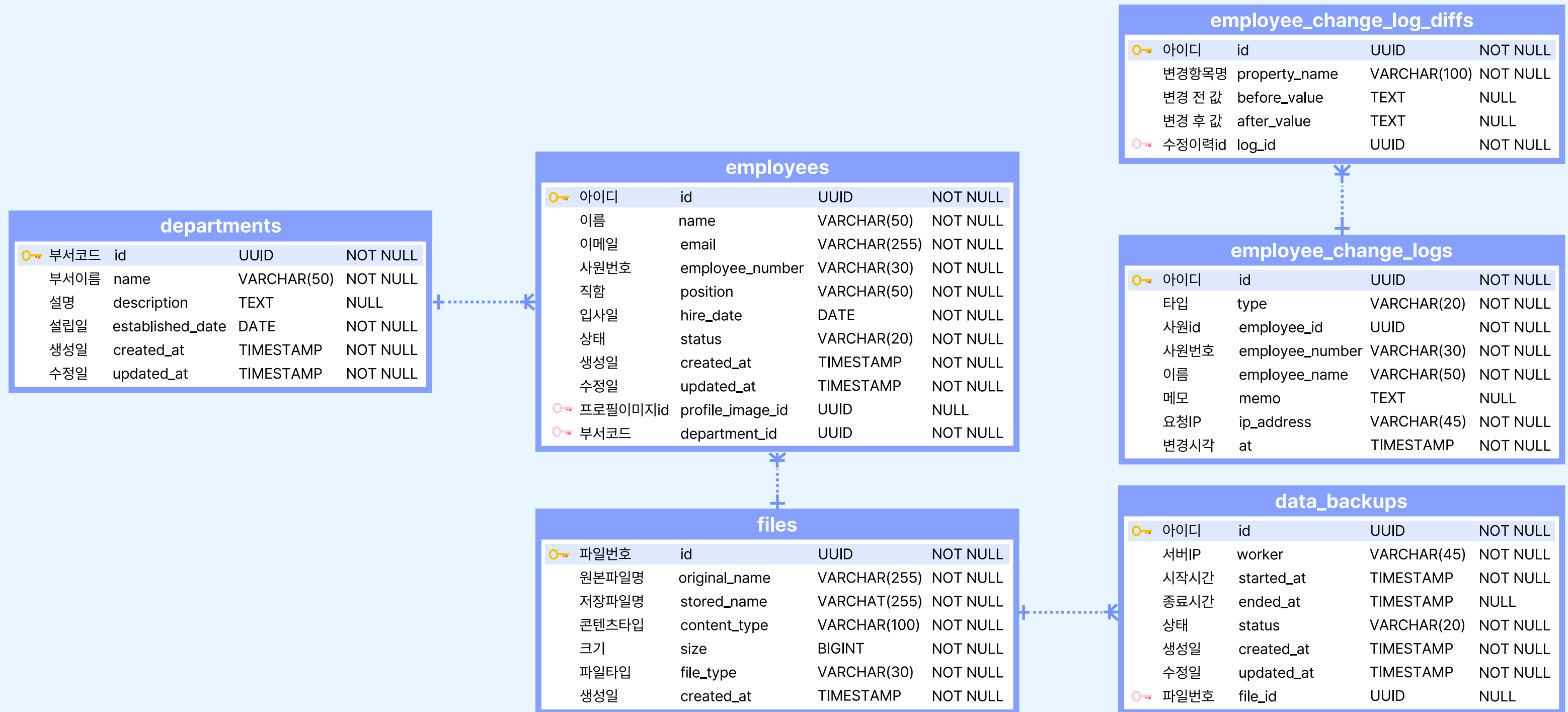
The screenshot displays four GitHub Project boards for the 'sb12-HR-Bank-teamYCC' project. The boards are categorized by status: '해야할 것' (To Do) with 2 items, '진행중' (In Progress) with 6 items, 'PR / 코드 리뷰 중' (Review) with 5 items, and '완료' (Done) with 15 items. Each board lists specific tasks with their IDs and descriptions, such as '[Docs] README 작성 및 발표 자료' and '[ChangeLog] 직원 CRUD에 이력 기록 연동'.

UI 흐름도

서비스의 전체 구조 파악을 위한 정보입니다.



ERD



시스템 아키텍처

인프라 관점

GitHub-Railway 연동으로 Spring Boot 서버를 배포하고,
업무 데이터와 실제 파일을 PostgreSQL·File Storage로 분리 관리합니다.

배포 관점



Developer

Push / PR / Merge



GitHub

자동 배포 Trigger



Railway
[환경 변수]

DB 접속 환경변수
파일 저장 경로
배치 실행 주기

App 빌드-배포
[환경 변수] Env 주입



Spring Boot
Application Server

REST API 처리
트랜잭션 기반 비즈니스 로직
백업 배치 실행
Swagger API 문서 제공

업무 데이터 조회·저장



Railway
PostgreSQL

직원 부서
직원 수정 이력
파일 메타데이터
데이터 백업 이력



File
Storage

프로필 이미지
백업 파일
에러 로그

파일 저장 / 조회

데이터 관점

REST API 요청

API 응답

대시보드

Dashboard

01 실시간 조회

총 직원 수
최근 변경 건수
이번 달 신규 입사자
마지막 백업

02 직원수 변동 추이

03 기간별 조회

04 그래프 상세

05 부서별 직원 분포

06 직무별 직원 분포

01

05

06

02

03

04

03

01 부서 검색

02 부서 추가하기

등록하기·취소

03 수정하기

04 삭제

01

HR Bank
인사관리시스템

총 10팀

부서명 또는 설명을 입력해주세요

02

부서 추가하기

부서명 *
부서명을 입력해주세요

설립일 *
날짜를 선택해주세요

설명 *
부서에 대한 설명을 입력해주세요

취소 등록하기

03

부서 관리

부서명 또는 설명을 입력해주세요

부서명	설명	인원수	부서생성일자	삭제	수정
보안팀	개인정보 보호와 시스템 보안을 담당합니다.	1001명	2023년 2월 1일	[X]	[Pencil]
품질관리팀	데이터 검수 및 품질 기준 관리를 담당합니다.	1001명	2022년 9월 20일	[X]	[Pencil]
데이터팀	데이터 수집, 정제, 분석 파이프라인을 담당합니다.	1000명	2022년 8월 10일	[X]	[Pencil]
마케팅팀	캠페인과 대외 커뮤니케이션을 담당합니다.	1001명	2022년 7월 15일	[X]	[Pencil]
디자인팀	UI/UX와 브랜드 디자인을 담당합니다.	1000명	2021년 12월 1일	[X]	[Pencil]
기획팀	서비스 정책과 프로젝트 기획을 담당합니다.	1000명	2021년 7월 12일	[X]	[Pencil]

04

! '보안팀' 부서를 삭제할까요?
삭제 후에는 되돌릴 수 없어요

취소 삭제하기

04

직원 관리

03

Employee

01 직원 검색

이름·이메일·상태
사번·부서명·직함·입사일

02 직원 추가하기

프로필 사진
이름·이메일
부서·직함·입사일
메모
등록하기·취소

03 수정하기

상태 *외 추가와 동일

04 삭제

01

HR Bank

총 10005명

이름 또는 이메일을 입력해주세요

상태

+

직원 등록하기

사번을 입력해주세요

부서명을 입력해주세요

직함을 입력해주세요

입사일을 선택해주세요

02

직원 등록하기

프로필 사진

이름 *
이름을 입력해주세요

이메일 *
이메일을 입력해주세요

부서
부서 입력 필수

직함 *
직함을 입력해주세요

입사일
날짜를 선택해주세요

메모 (변경 이력에 기록됨)
변경 사유나 메모를 입력해주세요

취소

등록하기

이름	사원번호	부서명	직함	입사일	재직상태
이현우 abc@abc.com	EMP-2026-10006	보안팀	사원	2026년 5월 23일	재직
김주훈 employee052254@example.com	EMP-2026-10003	데이터팀	사원		
안건호 employee0052149@example.com	EMP-2026-10005	개발팀	주임		
엄성현 employee261357@example.com	EMP-2026-10004	개발팀	사원		
김수아 employee051314@example.com	EMP-2026-10001	품질관리팀	팀장		

정보 수정하기

프로필 사진

이름 *
김주훈

이메일 *
employee052254@example.com

부서
데이터팀

직함 *
사원

입사일
2026년 5월 18일

상태
재직중

메모 (변경 이력에 기록됨)
변경 사유나 메모를 입력해주세요

취소

수정하기

04

!

'이현우' 직원을 삭제할까요?
삭제 후에는 되돌릴 수 없어요

취소

삭제하기

수정 이력

ChangeLog

01 수정 이력 검색

사번·유형

내용·IP 주소·수정 날짜

02 수정 이력 상세

프로필 사진

이름·사번

유형·일시·IP 주소

변경 상세 내용

- 변경 전

- 변경 후

01

총 30014건

Q 사번을 입력해주세요 유형  

내용을 입력해주세요 IP 주소를 입력해주세요  날짜를 선택해주세요

02

수정 이력 상세



**김주훈**
EMP-2026-10003

유형 • 정보 수정

일시 2026년 05월 22일 10:27:28

IP 주소 182.225.164.15

변경 상세 내용

변경 전

변경 후

마케팅팀

데이터팀

데이터 백업

 **HR Bank**
인사관리시스템

대시보드

부서 관리

직원 관리

수정 이력

데이터 백업

데이터 백업

총 385팀

작업자를 입력해주세요

상태 ▾

날짜를 선택해주세요

2026년 05월 22일 10:28:52

마지막 백업

백업하기

ID	작업자	시작시간 ▾	종료시간 ▴	작업상태 ▴	다운로드
019e4d4c-bcad-76dc-83f7-71798a465b0d	182.225.164.15	2026년 05월 22일 10:28:55	2026년 05월 22일 10:28:55	건너뛸	
019e4d4c-b8a4-7b70-b0b5-082ed19a9567	182.225.164.15	2026년 05월 22일 10:28:54	2026년 05월 22일 10:28:54	건너뛸	
019e4d4c-b044-7835-bdf0-4a2bb13e7bda	182.225.164.15	2026년 05월 22일 10:28:52	2026년 05월 22일 10:28:52	완료	다운로드
019e4d32-428d-76d6-a8da-fb5f52ab5ef2	system	2026년 05월 22일 10:00:00	2026년 05월 22일 10:00:00	완료	다운로드
019e4d0b-c2c9-7ab3-96cc-eeb803094d8b	112.165.74.220	2026년 05월 22일 09:17:56	2026년 05월 22일 09:17:57	완료	다운로드
019e4cfb-540c-766d-	svstem	2026년 05월 22일	2026년 05월 22일	건너뛸	

Data Backup

01 백업 이력 검색

작업자 IP
상태
백업 날짜

02 백업하기

03 다운로드

사원번호 동시 생성 문제

Problem

사원번호를 findMax + 1 방식으로 생성하는 도중에 새로운 직원 등록 요청이 들어오면 동시에 같은 사원번호를 조회할 수 있음. 이로 인해 동일한 번호가 생성되어 중복 저장 문제 발생

Solution

마지막 사원번호를 조회를 조회할 때 pessimistic lock을 사용해 동시 접근 방지

```
public List<Employee> findLatestByEmployeeNumberPrefixForUpdate(String prefix, Pageable pageable) {  
    return queryFactory  
        .selectFrom(employee)  
        .where(employee.employeeNumber.startsWith(prefix))  
        .orderBy(employee.employeeNumber.desc())  
        .offset(pageable.getOffset())  
        .limit(pageable.getPageSize())  
        .setLockMode(LockModeType.PESSIMISTIC_WRITE)  
        .fetch();  
}
```

데이터 백업 중 OOM 발생

Problem

전체 직원 데이터를 한번에 조회해 CSV 파일로 저장할 경우,
데이터가 메모리에 누적되어 OOM 발생 가능성 존재

```
private static final int CHUNK_SIZE = 1000;
```

Solution

데이터를 1000건 단위 청크로 분할 조회하고 순차적으로
CSV에 저장해 메모리 사용량을 분산하고 데이터 누적을 방지

```
int page = 0;
Page<Employee> chunk;
do {
    chunk = employeeRepository.findAllWithDepartmentForBackup(
        PageRequest.of(page++, CHUNK_SIZE)
    );
    for (Employee e : chunk.getContent()) {
        writer.write(toCsvRow(e));
        writer.newLine();
    }
} while (chunk.hasNext());
```

데이터 백업 상태 지연 문제

Problem

데이터 백업 시작 시 상태를 IN_PROGRESS로 저장하였으나,
API 조회에서는 항상 COMPLETED 상태만 확인되는 문제 발생

```
// -----  
// 데이터 백업 프로세스 (STEP 1 ~ 4)  
// -----  
  
public DataBackupDto backup(String worker) { 2개 사용 위치  Ⓜ gusdn6763  
  
    // STEP.1: 백업 필요 여부 판단  
    if (!isBackupNeeded()) {...}  
  
    // STEP.2: IN_PROGRESS 이력 등록  
    DataBackup backup = dataBackupTxService.createInProgressBackup(worker);
```

Solution

@Transactional이 프록시 기반으로 동작하는 점을 고려해
백업 상태 저장 로직을 별도 프록시 클래스로 분리하고
트랜잭션 경계를 분리

```
public class DataBackupTxService {  
  
    private final DataBackupRepository dataBackupRepository;  
  
    @Transactional(propagation = Propagation.REQUIRES_NEW) 1개 사용 위치  
    public DataBackup createInProgressBackup(String worker) {  
        return dataBackupRepository.save(  

```

N+1 문제

Problem

부서 목록 조회 시 각 부서의 직원 수를 개별 조회해,
부서 수만큼 추가 쿼리가 발생하는 N+1 문제 발생

QueryDSL을 사용해
부서와 직원 수를 그룹화하여
한번의 쿼리로 조회하도록 개선

Solution

```
return queryFactory.select(Projections.constructor(
    DepartmentResponse.class,
    department.id,
    department.name,
    department.description,
    department.establishedDate,
    employee.count().intValue()))
    .from(department)
    .leftJoin(employee)
    .on(employee.department.id.eq(department.id))
    .where(keywordCondition)
    .groupBy(department.id)
    .fetch();
```

THANK YOU

TEAM 5 | YCC

HR BANK